

□ **Suggerimento:** Collegamento Tematico

I concetti di ottimizzazione qui trattati si applicano anche al training delle reti neurali dove si usa Gradient Descent. Il teorema No Free Lunch vale per qualsiasi algoritmo di ottimizzazione, inclusi quelli del deep learning.

## Ottimizzatori Globali

Gli algoritmi di ottimizzazione esaminati per la soluzione dei problemi di segmentazione e registrazione di immagini visti finora hanno la caratteristica di essere ottimizzatori locali. Questo implica che la soluzione trovata dipende fortemente dalle condizioni iniziali. Chiaramente sarebbe preferibile avere degli ottimizzatori globali, cioè degli algoritmi in grado di massimizzare una funzione indipendentemente dalle condizioni iniziali. Nella pratica si rileva che gli ottimizzatori globali hanno prestazioni peggiori degli ottimizzatori locali, sia come capacità di trovare una soluzione migliore al problema che come tempo di calcolo. Ad esempio un algoritmo a ricerca esaustiva (o random search) è sicuramente un ottimizzatore globale in quanto non ha condizioni iniziali ma ha complessità computazionale molto alta. Queste considerazioni si formalizzano nel cosiddetto teorema “no free lunch” (niente per niente). Il teorema afferma che “Tutti gli algoritmi che ricercano il massimo di una funzione hanno esattamente la stessa prestazione media quando vengono provati su tutte le possibili funzioni”. La figura illustra la differenza tra un algoritmo generico ed uno specializzato. In termini formali, definiamo un algoritmo  $a$  come una funzione che partendo da un insieme di punti  $m$  nello spazio delle soluzioni produce un nuovo punto  $d$  con un certo valore  $y$  della funzione da ottimizzare  $f$ . Allora:

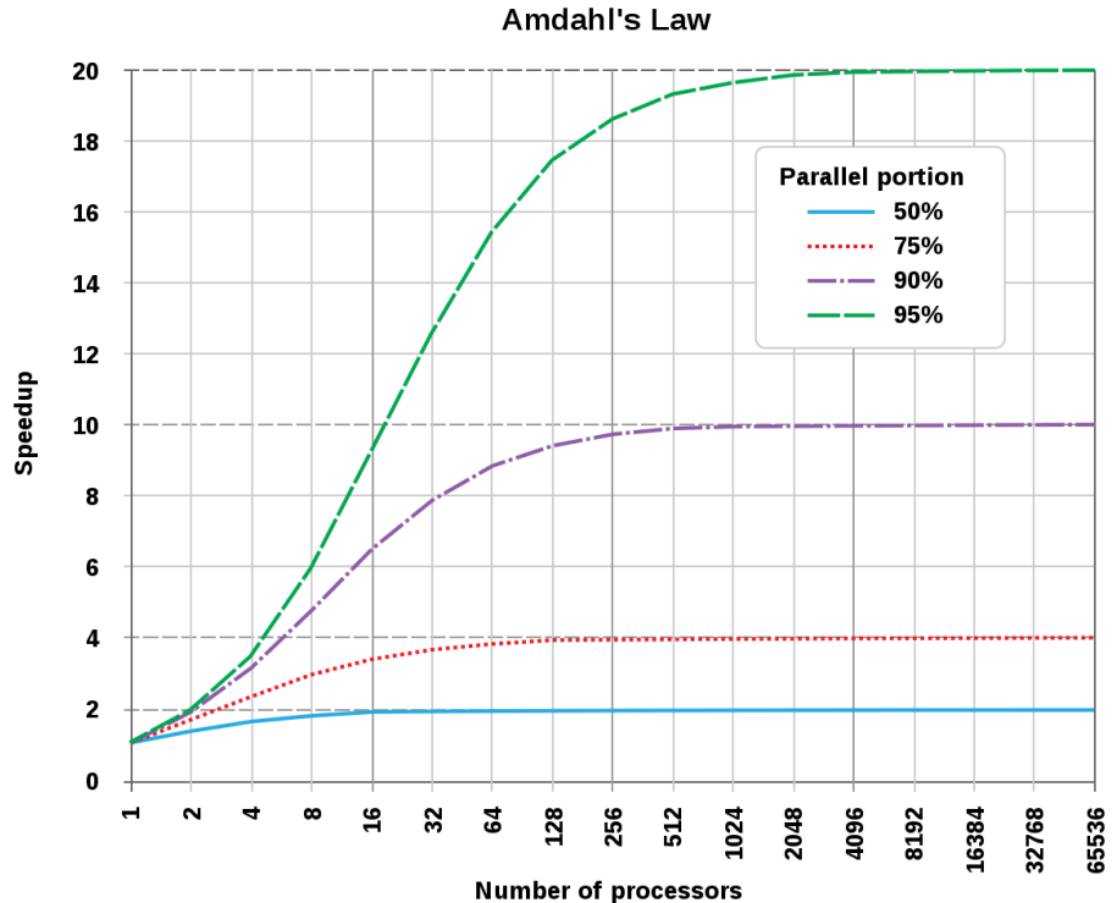
$$\sum_f P(d_m^y \mid f, m, a_1) = \sum_f P(d_m^y \mid f, m, a_2)$$

Dove  $P(d_m^y \mid f, m, a_1)$  è la probabilità di ottenere un punto nello spazio delle soluzioni con valore  $y$  condizionato dalla funzione da ottimizzare, dal numero di iterazioni  $m$  e dall'algoritmo usato  $a$ . Il teorema è valido per ogni possibile coppia di algoritmi  $a_1$  e  $a_2$ . Il teorema è stato dimostrato da Wolpert e Macready nel 1997. Dal teorema si deduce come corollario che per ogni possibile misura di prestazioni  $M(d_m^y)$ , cioè ogni misura che ci dice quanto è “buona” la stima  $d_m^y$  per il problema che vogliamo risolvere, la media su tutte le  $f$  della  $P(M(d_m^y) \mid f, m, a)$  è indipendente da  $a$ . Il teorema afferma quindi che per ogni misura di prestazioni, la media delle prestazioni su tutte le funzioni possibili è indipendente dall'algoritmo. Quindi se si specializza l'algoritmo su di una particolare classe di funzioni, si otterranno prestazioni peggiori su tutte le funzioni non appartenenti alla classe selezionata. Il teorema è molto generale e vale per tutti i tipi di algoritmi. Un modo equivalente di vedere la cosa è affermare che un algoritmo migliore della ricerca casuale su di un set di funzioni incorpora informazioni esplicite o implicite sul problema da risolvere. Tuttavia, sull'insieme globale delle funzioni possibili, le prestazioni medie di qualsiasi algoritmo sono equivalenti a quelle di un algoritmo di ricerca casuale. Un modo

di realizzare un ottimizzatore globale è quello di utilizzare un ottimizzatore locale variando le sue condizioni iniziali. Dato un search-space  $S$ , supponiamo di avere un ottimizzatore locale in grado di convergere ad una soluzione locale  $A$  se le condizioni iniziali sono in uno spazio  $dS$  intorno ad  $A$ . Se dividiamo  $S$  in  $N$  parti tali che ogni singola parte sia più piccola di  $dS$ , possiamo sperare di trovare la soluzione ottima eseguendo  $N$  volte l'ottimizzatore locale. Ad esempio in un algoritmo di clustering possiamo ripetere il procedimento partendo ogni volta da condizioni diverse, come avviene attraverso l'opzione "replicate" del kmeans. In generale questo approccio è detto MultiStart ed è implementato in Matlab dalla funzione omonima. Definito il numero  $N$  di repliche dell'ottimizzatore locale, il tempo di calcolo essendo i processi di ottimizzazione indipendenti sarà mediamente  $N$  volte il tempo di una singola ottimizzazione. Essendo però i processi di ottimizzazione indipendenti, è possibile farli girare in parallelo su più processori (o processori virtuali, i cosiddetti core nelle architetture moderne), ottenendo un tempo di calcolo  $N/C$  dove  $C$  è il numero di core. La scelta delle condizioni iniziali può essere fatta dividendo in regioni il search-space o generandole in modo casuale. In generale in un ottimizzatore globale, essendo il tempo di calcolo elevato, è fondamentale che l'algoritmo sia parallelizzabile o scalabile, cioè che sia in grado di girare in modo efficiente su una macchina parallela. Idealmente, il tempo di calcolo di un algoritmo parallelizzabile dovrebbe scalare linearmente con il numero di processori. Di fatto lo speedup (cioè l'incremento di velocità ottenuto attraverso l'uso di una architettura parallela) sarà dato dalla legge di Amdahl:

$$S = \frac{1}{1 - p + \frac{p}{s}}$$

dove  $p$  è la percentuale del tempo di calcolo della parte parallelizzabile dell'algoritmo (calcolata sull'esecuzione sequenziale dell'algoritmo stesso) e  $s$  è lo speedup che si ottiene facendo girare la parte parallelizzabile sulla macchina parallela. Se  $p=1$ , cioè tutto l'algoritmo è parallelizzabile,  $S=s$  per cui lo speedup globale è uguale all'efficienza del processo di parallelizzazione. Se  $p<1$ , come avviene sempre in pratica, anche nel caso ideale  $s=N$  ( $N$  = numero di processori), per  $N$  molto grande avremo  $S=1/(1-p)$ , per cui lo speedup è limitato dalla parte di algoritmo non



parallelizzabile.

Come si osserva in figura (tratta da Wikipedia English, voce Parallel Computing), basta che una piccola parte dell'algoritmo non sia parallelizzabile per limitare in modo sostanziale lo speedup e rendere inutile l'uso di un numero elevato di processori. Tipicamente la parte non parallelizzabile di un algoritmo include l'accesso ai dati (da disco o memoria) e la raccolta dei risultati dai processi paralleli. Ad esempio nel MultiStart dobbiamo leggere i dati e far partire i processi di ottimizzazione locale ed alla fine ordinare i risultati secondo i valori ottenuti e scegliere quello ottimo. Nella pratica quindi  $p$  dipende dalla quantità di dati (più dati devo analizzare maggiore è il tempo che impiega la parte parallela dell'algoritmo rispetto a quella sequenziale), per cui la legge di Amdahl si traduce nel fatto che ha senso utilizzare un numero elevato di processori solo per elaborare volumi di dati molto grandi (legge di Gustafson).

---

□ **Suggerimento:** Collegamento Tematico

I concetti di ottimizzazione qui trattati si applicano anche al training delle reti neurali dove si usa Gradient Descent. Il teorema No Free Lunch vale per qualsiasi algoritmo di ottimizzazione, inclusi quelli del deep learning.